

CAMPUS: Madureira

CURSO: Desenvolvimento Full Stack

DISCIPLINA: DGT2821 – Programação Back-End com Java

TURMA: 9001

SEMESTRE LETIVO: 2026.2

ALUNO: Christian Marques de Oliveira Silva

GIT-HUB: https://github.com/christian-marques/DGT2821_Programacao_BackEnd_Java

TRABALHO PRÁTICO

Implementação de um cadastro de clientes em modo texto, com persistência em arquivos, baseado na tecnologia Java

Sumário

1.	Criação das Entidades e Sistema de Persistência	3
1.1.	Objetivo da Prática	3
1.2.	Códigos	3
1.3.	Resultados	10
1.4.	Análise e Conclusão	11
2.	Criação do Cadastro em Modo Texto	12
2.1.	Objetivo da Prática	12
2.2.	Códigos	12
2.3.	Resultados	18
2.4.	Análise e Conclusão	21

Lista de Imagens

Imagem 1 – Resultado 1 no NetBeans (tela cheia)	10
Imagem 2 – Resultado 1 (output)	11
Imagem 3 - Cadastro de Pessoa Física e Jurídica	18
Imagem 4 - Exibição de Pessoa Física e Jurídica.....	18
Imagem 5 - Alteração de Pessoa.....	19
Imagem 6 - Exibir pelo ID	19
Imagem 7 - Excluir pelo ID.....	19
Imagem 8 - Salvando os dados em arquivo externo	20
Imagem 9 - Finalização do programa	20
Imagem 10 - Recuperação dos dados salvos em arquivo externo.....	20

1. Criação das Entidades e Sistema de Persistência

1.1. Objetivo da Prática

Desenvolver um sistema de cadastro em Java, aplicando conceitos de herança, polimorfismo e encapsulamento. Também foi trabalhada a persistência e recuperação de dados em arquivos binários.

1.2. Códigos

- CadastroPOO.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Main.java to edit this
template
 */
package cadastropoo;

import java.io.IOException;
import model.PessoaFisica;
import model.PessoaFisicaRepo;
import model.PessoaJuridica;
import model.PessoaJuridicaRepo;

/**
 *
 * @author chris
 */
public class CadastroPOO {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {

        try {
            // Repositório de pessoas físicas
            PessoaFisicaRepo repo1 = new PessoaFisicaRepo();

            repo1.inserir(
                new PessoaFisica(
                    1,
                    "Ada Marques",
                    "111.111.111-11",
                    25
                )
            );

            repo1.inserir(
                new PessoaFisica(
                    2,
                    "Luna Marques",
                    "222.222.222-22",
                    32
                )
            );

            repo1.persistir("pessoas_fisicas.bin");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

```

        System.out.println("Dados de Pessoa Física Armazenados.");

        // Recuperação das pessoas físicas
        PessoaFisicaRepo repo2 = new PessoaFisicaRepo();

        repo2.recuperar("pessoas_fisicas.bin");

        System.out.println("Dados de Pessoa Física Recuperados.\n");

        for (PessoaFisica pessoa : repo2.obterTodos()) {
            pessoa.exibir();
            System.out.println();
        }

        // Repositório de pessoas jurídicas
        PessoaJuridicaRepo repo3 = new PessoaJuridicaRepo();

        repo3.inserir(
            new PessoaJuridica(
                1,
                "Adidas",
                "11.111.111/0001-11"
            )
        );

        repo3.inserir(
            new PessoaJuridica(
                2,
                "Nike",
                "22.222.222/0001-22"
            )
        );

        repo3.persistir("pessoas_juridicas.bin");

        System.out.println("Dados de Pessoa Jurídica Armazenados.");

        // Recuperação das pessoas jurídicas
        PessoaJuridicaRepo repo4 = new PessoaJuridicaRepo();

        repo4.recuperar("pessoas_juridicas.bin");

        System.out.println("Dados de Pessoa Jurídica Recuperados.\n");

        for (PessoaJuridica pessoa : repo4.obterTodos()) {
            pessoa.exibir();
            System.out.println();
        }

    } catch (IOException | ClassNotFoundException erro) {
        System.out.println(
            "Erro ao salvar ou recuperar os dados: "
            + erro.getMessage()
        );
    }
}

```

- Pessoa.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */

package model;

import java.io.Serializable;

/**
 *
 * @author chris
 */
public class Pessoa implements Serializable {

    private static final long serialVersionUID = 1L;

    private int id;
    private String nome;

    public Pessoa() {
    }

    public Pessoa(int id, String nome) {
        this.id = id;
        this.nome = nome;
    }

    public void exibir() {
        System.out.println("Id: " + id);
        System.out.println("Nome: " + nome);
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }
}
```

- PessoaFisica.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */
package model;

import java.io.Serializable;

/**
 *
 * @author chris
 */
public class PessoaFisica extends Pessoa implements Serializable {

    private static final long serialVersionUID = 1L;

    private String cpf;
    private int idade;

    public PessoaFisica() {
    }

    public PessoaFisica(int id, String nome, String cpf, int idade) {
        super(id, nome);
        this.cpf = cpf;
        this.idade = idade;
    }

    @Override
    public void exibir() {
        super.exibir();
        System.out.println("CPF: " + cpf);
        System.out.println("Idade: " + idade);
    }

    public String getCpf() {
        return cpf;
    }

    public void setCpf(String cpf) {
        this.cpf = cpf;
    }

    public int getIdade() {
        return idade;
    }

    public void setIdade(int idade) {
        this.idade = idade;
    }
}
```

- PessoaFisicaRepo.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */
package model;

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.util.ArrayList;

/**
 *
 * @author chris
 */
public class PessoaFisicaRepo {

    private ArrayList<PessoaFisica> pessoasFisicas;

    public PessoaFisicaRepo() {
        pessoasFisicas = new ArrayList<>();
    }

    public void inserir(PessoaFisica pessoaFisica) {
        pessoasFisicas.add(pessoaFisica);
    }

    public void alterar(PessoaFisica pessoaFisica) {
        PessoaFisica pessoaEncontrada = obter(pessoaFisica.getId());

        if (pessoaEncontrada != null) {
            pessoaEncontrada.setNome(pessoaFisica.getNome());
            pessoaEncontrada.setCpf(pessoaFisica.getCpf());
            pessoaEncontrada.setIdade(pessoaFisica.getIdade());
        }
    }

    public void excluir(int id) {
        PessoaFisica pessoaEncontrada = obter(id);

        if (pessoaEncontrada != null) {
            pessoasFisicas.remove(pessoaEncontrada);
        }
    }

    public PessoaFisica obter(int id) {
        return pessoasFisicas.stream()
            .filter(pessoa -> pessoa.getId() == id)
            .findFirst()
            .orElse(null);
    }

    public ArrayList<PessoaFisica> obterTodos() {
        return pessoasFisicas;
    }
}
```

```

    public void persistir(String nomeArquivo) throws IOException {
        try (FileOutputStream arquivo = new FileOutputStream(nomeArquivo);
             ObjectOutputStream objeto = new ObjectOutputStream(arquivo)) {

            objeto.writeObject(pessoasFisicas);
        }
    }

    @SuppressWarnings("unchecked")
    public void recuperar(String nomeArquivo)
        throws IOException, ClassNotFoundException {

        try (FileInputStream arquivo = new FileInputStream(nomeArquivo);
             ObjectInputStream objeto = new ObjectInputStream(arquivo)) {

            pessoasFisicas = (ArrayList<PessoaFisica>) objeto.readObject();
        }
    }
}

```

- PessoaJuridica.java

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */
package model;

import java.io.Serializable;

/**
 *
 * @author chris
 */
public class PessoaJuridica extends Pessoa implements Serializable {

    private static final long serialVersionUID = 1L;

    private String cnpj;

    public PessoaJuridica() {
    }

    public PessoaJuridica(int id, String nome, String cnpj) {
        super(id, nome);
        this.cnpj = cnpj;
    }

    @Override
    public void exibir() {
        super.exibir();
        System.out.println("CNPJ: " + cnpj);
    }

    public String getCnpj() {
        return cnpj;
    }
}

```

```

    public void setCnpj(String cnpj) {
        this.cnpj = cnpj;
    }
}

```

- PessoaJuridicaRepo.java

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */
package model;

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.util.ArrayList;

/**
 *
 * @author chris
 */
public class PessoaJuridicaRepo {

    private ArrayList<PessoaJuridica> pessoasJuridicas;

    public PessoaJuridicaRepo() {
        pessoasJuridicas = new ArrayList<>();
    }

    public void inserir(PessoaJuridica pessoaJuridica) {
        pessoasJuridicas.add(pessoaJuridica);
    }

    public void alterar(PessoaJuridica pessoaJuridica) {
        PessoaJuridica pessoaEncontrada = obter(pessoaJuridica.getId());

        if (pessoaEncontrada != null) {
            pessoaEncontrada.setNome(pessoaJuridica.getNome());
            pessoaEncontrada.setCnpj(pessoaJuridica.getCnpj());
        }
    }

    public void excluir(int id) {
        PessoaJuridica pessoaEncontrada = obter(id);

        if (pessoaEncontrada != null) {
            pessoasJuridicas.remove(pessoaEncontrada);
        }
    }

    public PessoaJuridica obter(int id) {
        return pessoasJuridicas.stream()
            .filter(pessoa -> pessoa.getId() == id)

```

```

        .findFirst()
        .orElse(null);
    }

    public ArrayList<PessoaJuridica> obterTodos() {
        return pessoasJuridicas;
    }

    public void persistir(String nomeArquivo) throws IOException {
        try (FileOutputStream arquivo = new FileOutputStream(nomeArquivo);
             ObjectOutputStream objeto = new ObjectOutputStream(arquivo)) {

            objeto.writeObject(pessoasJuridicas);
        }
    }

    @SuppressWarnings("unchecked")
    public void recuperar(String nomeArquivo)
        throws IOException, ClassNotFoundException {

        try (FileInputStream arquivo = new FileInputStream(nomeArquivo);
             ObjectInputStream objeto = new ObjectInputStream(arquivo)) {

            pessoasJuridicas =
                (ArrayList<PessoaJuridica>) objeto.readObject();
        }
    }
}

```

1.3. Resultados

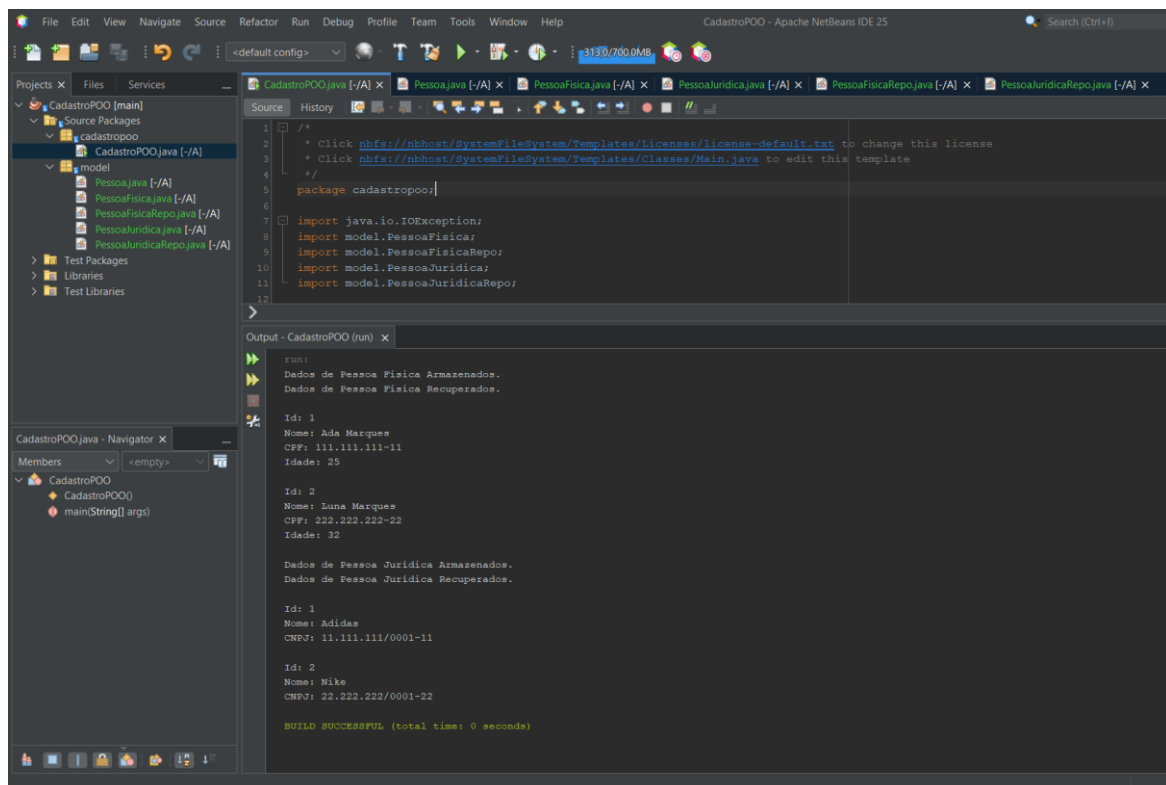
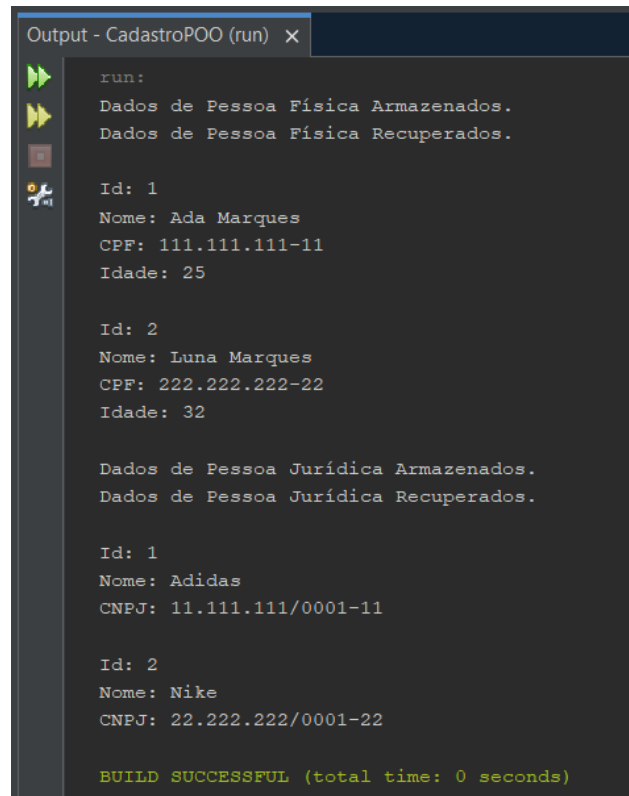


Imagem 1 – Resultado 1 no NetBeans (tela cheia)



```
Output - CadastroPOO (run) X
run:
Dados de Pessoa Física Armazenados.
Dados de Pessoa Física Recuperados.

Id: 1
Nome: Ada Marques
CPF: 111.111.111-11
Idade: 25

Id: 2
Nome: Luna Marques
CPF: 222.222.222-22
Idade: 32

Dados de Pessoa Jurídica Armazenados.
Dados de Pessoa Jurídica Recuperados.

Id: 1
Nome: Adidas
CNPJ: 11.111.111/0001-11

Id: 2
Nome: Nike
CNPJ: 22.222.222/0001-22

BUILD SUCCESSFUL (total time: 0 seconds)
```

Imagem 2 – Resultado 1 (output)

1.4. Análise e Conclusão

- Quais as vantagens e desvantagens do uso de herança?

A herança evita a repetição de código e permite reaproveitar atributos e métodos de uma classe principal. Como desvantagem, pode deixar as classes muito dependentes e dificultar alterações no projeto.

- Por que a interface *Serializable* é necessária ao efetuar persistência em arquivos binários?

A interface *Serializable* permite transformar os objetos em uma sequência de bytes. Assim, eles podem ser salvos em arquivos binários e recuperados posteriormente.

- Como o paradigma funcional é utilizado pela API *stream* no Java?

A API *Stream* utiliza funções para filtrar, buscar, ordenar e processar dados de uma coleção. Nesta prática, ela foi usada para localizar uma pessoa pelo id.

- Quando trabalhamos com Java, qual padrão de desenvolvimento é adotado na persistência de dados em arquivos?

É utilizado o padrão *Repository*, semelhante ao *DAO*, que separa o acesso e a persistência dos dados das demais partes do sistema. Isso deixa o código mais organizado e facilita sua manutenção.

2. Criação do Cadastro em Modo Texto

2.1. Objetivo da Prática

Desenvolvimento de um sistema de cadastro em modo texto, permitindo incluir, alterar, excluir, consultar, salvar e recuperar dados de pessoas físicas e jurídicas. Também foi aplicado o tratamento de exceções na persistência dos arquivos.

2.2. Códigos

- CadastroPOO.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
 this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Main.java to edit this
 template
 */
package cadastropoo;

import java.io.IOException;
import java.util.Scanner;
import model.PessoaFisica;
import model.PessoaFisicaRepo;
import model.PessoaJuridica;
import model.PessoaJuridicaRepo;

/**
 *
 * @author chris
 */
public class CadastroP00 {

    private static final Scanner entrada = new Scanner(System.in);

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {

        PessoaFisicaRepo repoFisica = new PessoaFisicaRepo();
        PessoaJuridicaRepo repoJuridica = new PessoaJuridicaRepo();

        int opcao;

        do {
            exibirMenu();
            opcao = lerInteiro("Opção: ");

            switch (opcao) {
                case 1:
                    incluirPessoa(repoFisica, repoJuridica);
                    break;

                case 2:
                    alterarPessoa(repoFisica, repoJuridica);
                    break;

                case 3:
                    excluirPessoa(repoFisica, repoJuridica);
```

```

        break;

    case 4:
        exibirPorId(repoFisica, repoJuridica);
        break;

    case 5:
        exibirTodos(repoFisica, repoJuridica);
        break;

    case 6:
        salvarDados(repoFisica, repoJuridica);
        break;

    case 7:
        recuperarDados(repoFisica, repoJuridica);
        break;

    case 0:
        System.out.println("Programa finalizado.");
        break;

    default:
        System.out.println("Opção inválida.");
        break;
    }

    } while (opcao != 0);

    entrada.close();
}

private static void exibirMenu() {

    System.out.println();
    System.out.println("1-Incluir Pessoa");
    System.out.println("2-Alterar Pessoa");
    System.out.println("3-Excluir Pessoa");
    System.out.println("4-Exibir pelo Id");
    System.out.println("5-Exibir Todos");
    System.out.println("6-Salvar Dados");
    System.out.println("7-Recuperar Dados");
    System.out.println("0-Finalizar Programa");
}

private static void incluirPessoa(
    PessoaFisicaRepo repoFisica,
    PessoaJuridicaRepo repoJuridica) {

    char tipo = lerTipo();
    int id = lerInteiro("Id: ");
    String nome = lerTexto("Nome: ");

    if (tipo == 'F') {
        String cpf = lerTexto("CPF: ");
        int idade = lerInteiro("Idade: ");

        PessoaFisica pessoaFisica
            = new PessoaFisica(id, nome, cpf, idade);

        repoFisica.inserir(pessoaFisica);
    }
}

```

```

    } else {
        String cnpj = lerTexto("CNPJ: ");

        PessoaJuridica pessoaJuridica
            = new PessoaJuridica(id, nome, cnpj);

        repoJuridica.inserir(pessoaJuridica);
    }

    System.out.println("Pessoa incluída com sucesso.");
}

private static void alterarPessoa(
    PessoaFisicaRepo repoFisica,
    PessoaJuridicaRepo repoJuridica) {

    char tipo = lerTipo();
    int id = lerInteiro("Informe o Id: ");

    if (tipo == 'F') {
        PessoaFisica pessoaAtual = repoFisica.obter(id);

        if (pessoaAtual == null) {
            System.out.println("Pessoa física não encontrada.");
            return;
        }

        System.out.println("\nDados atuais:");
        pessoaAtual.exibir();

        System.out.println("\nInforme os novos dados:");
        String nome = lerTexto("Nome: ");
        String cpf = lerTexto("CPF: ");
        int idade = lerInteiro("Idade: ");

        PessoaFisica pessoaNova
            = new PessoaFisica(id, nome, cpf, idade);

        repoFisica.alterar(pessoaNova);
    } else {
        PessoaJuridica pessoaAtual = repoJuridica.obter(id);

        if (pessoaAtual == null) {
            System.out.println("Pessoa jurídica não encontrada.");
            return;
        }

        System.out.println("\nDados atuais:");
        pessoaAtual.exibir();

        System.out.println("\nInforme os novos dados:");
        String nome = lerTexto("Nome: ");
        String cnpj = lerTexto("CNPJ: ");

        PessoaJuridica pessoaNova
            = new PessoaJuridica(id, nome, cnpj);

        repoJuridica.alterar(pessoaNova);
    }
}

```

```

        System.out.println("Pessoa alterada com sucesso.");
    }

    private static void excluirPessoa(
        PessoaFisicaRepo repoFisica,
        PessoaJuridicaRepo repoJuridica) {

        char tipo = lerTipo();
        int id = lerInteiro("Informe o Id: ");

        if (tipo == 'F') {
            PessoaFisica pessoaFisica = repoFisica.obter(id);

            if (pessoaFisica == null) {
                System.out.println("Pessoa física não encontrada.");
                return;
            }

            repoFisica.excluir(id);
        } else {
            PessoaJuridica pessoaJuridica = repoJuridica.obter(id);

            if (pessoaJuridica == null) {
                System.out.println("Pessoa jurídica não encontrada.");
                return;
            }

            repoJuridica.excluir(id);
        }

        System.out.println("Pessoa excluída com sucesso.");
    }

    private static void exibirPorId(
        PessoaFisicaRepo repoFisica,
        PessoaJuridicaRepo repoJuridica) {

        char tipo = lerTipo();
        int id = lerInteiro("Informe o Id: ");

        if (tipo == 'F') {
            PessoaFisica pessoaFisica = repoFisica.obter(id);

            if (pessoaFisica != null) {
                pessoaFisica.exibir();
            } else {
                System.out.println("Pessoa física não encontrada.");
            }
        } else {
            PessoaJuridica pessoaJuridica = repoJuridica.obter(id);

            if (pessoaJuridica != null) {
                pessoaJuridica.exibir();
            } else {
                System.out.println("Pessoa jurídica não encontrada.");
            }
        }
    }
}

```

```

private static void exibirTodos(
    PessoaFisicaRepo repoFisica,
    PessoaJuridicaRepo repoJuridica) {

    char tipo = lerTipo();

    if (tipo == 'F') {

        if (repoFisica.obterTodos().isEmpty()) {
            System.out.println(
                "Não existem pessoas físicas cadastradas."
            );
            return;
        }

        for (PessoaFisica pessoaFisica
            : repoFisica.obterTodos()) {

            pessoaFisica.exibir();
            System.out.println();
        }

    } else {

        if (repoJuridica.obterTodos().isEmpty()) {
            System.out.println(
                "Não existem pessoas jurídicas cadastradas."
            );
            return;
        }

        for (PessoaJuridica pessoaJuridica
            : repoJuridica.obterTodos()) {

            pessoaJuridica.exibir();
            System.out.println();
        }

    }
}

private static void salvarDados(
    PessoaFisicaRepo repoFisica,
    PessoaJuridicaRepo repoJuridica) {

    String prefixo = lerTexto("Prefixo dos arquivos: ");

    try {
        repoFisica.persistir(prefixo + ".fisica.bin");
        repoJuridica.persistir(prefixo + ".juridica.bin");

        System.out.println("Dados salvos com sucesso.");

    } catch (IOException erro) {
        System.out.println(
            "Erro ao salvar os dados: " + erro.getMessage()
        );
    }
}

private static void recuperarDados(

```

```

        PessoaFisicaRepo repoFisica,
        PessoaJuridicaRepo repoJuridica) {

    String prefixo = lerTexto("Prefixo dos arquivos: ");

    try {
        repoFisica.recuperar(prefixo + ".fisica.bin");
        repoJuridica.recuperar(prefixo + ".juridica.bin");

        System.out.println("Dados recuperados com sucesso.");

    } catch (IOException | ClassNotFoundException erro) {
        System.out.println(
            "Erro ao recuperar os dados: " + erro.getMessage()
        );
    }
}

private static char lerTipo() {

    while (true) {
        String tipo = lerTexto(
            "F-Pessoa Física | J-Pessoa Jurídica: "
        ).toUpperCase();

        if (tipo.equals("F") || tipo.equals("J")) {
            return tipo.charAt(0);
        }

        System.out.println(
            "Tipo inválido. Digite somente F ou J."
        );
    }
}

private static int lerInteiro(String mensagem) {

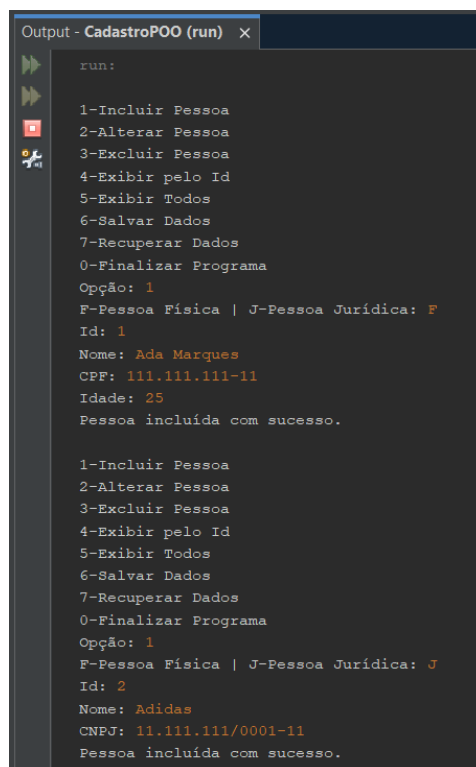
    while (true) {
        try {
            System.out.print(mensagem);
            return Integer.parseInt(entrada.nextLine());

        } catch (NumberFormatException erro) {
            System.out.println(
                "Digite um número inteiro válido."
            );
        }
    }
}

private static String lerTexto(String mensagem) {
    System.out.print(mensagem);
    return entrada.nextLine();
}
}

```

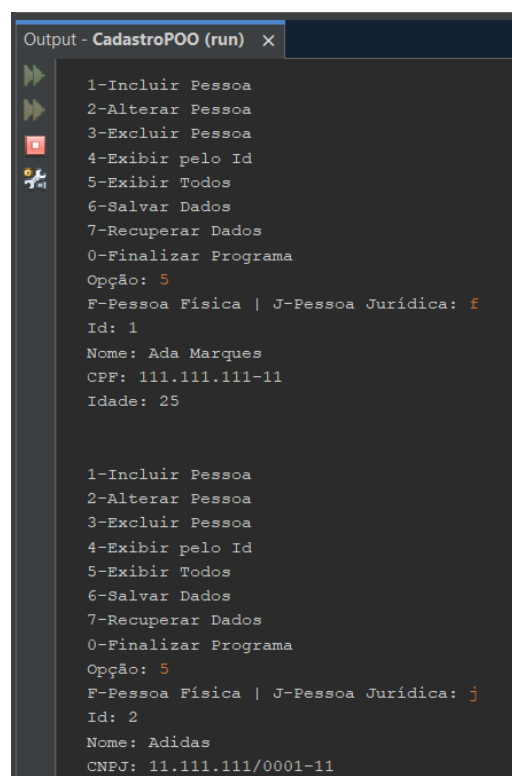
2.3. Resultados



```
Output - CadastroPOO (run) x
run:
1-Incluir Pessoa
2-Alterar Pessoa
3-Excluir Pessoa
4-Exibir pelo Id
5-Exibir Todos
6-Salvar Dados
7-Recuperar Dados
0-Finalizar Programa
Opção: 1
F-Pessoa Física | J-Pessoa Jurídica: F
Id: 1
Nome: Ada Marques
CPF: 111.111.111-11
Idade: 25
Pessoa incluída com sucesso.

1-Incluir Pessoa
2-Alterar Pessoa
3-Excluir Pessoa
4-Exibir pelo Id
5-Exibir Todos
6-Salvar Dados
7-Recuperar Dados
0-Finalizar Programa
Opção: 1
F-Pessoa Física | J-Pessoa Jurídica: J
Id: 2
Nome: Adidas
CNPJ: 11.111.111/0001-11
Pessoa incluída com sucesso.
```

Imagem 3 - Cadastro de Pessoa Física e Jurídica



```
Output - CadastroPOO (run) x
1-Incluir Pessoa
2-Alterar Pessoa
3-Excluir Pessoa
4-Exibir pelo Id
5-Exibir Todos
6-Salvar Dados
7-Recuperar Dados
0-Finalizar Programa
Opção: 5
F-Pessoa Física | J-Pessoa Jurídica: f
Id: 1
Nome: Ada Marques
CPF: 111.111.111-11
Idade: 25
Pessoa incluída com sucesso.

1-Incluir Pessoa
2-Alterar Pessoa
3-Excluir Pessoa
4-Exibir pelo Id
5-Exibir Todos
6-Salvar Dados
7-Recuperar Dados
0-Finalizar Programa
Opção: 5
F-Pessoa Física | J-Pessoa Jurídica: j
Id: 2
Nome: Adidas
CNPJ: 11.111.111/0001-11
Pessoa incluída com sucesso.
```

Imagem 4 - Exibição de Pessoa Física e Jurídica

```
Output - CadastroPOO (run) X
1-Incluir Pessoa
2-Alterar Pessoa
3-Excluir Pessoa
4-Exibir pelo Id
5-Exibir Todos
6-Salvar Dados
7-Recuperar Dados
0-Finalizar Programa
Opção: 2
F-Pessoa Física | J-Pessoa Jurídica: J
Informe o Id: 2

Dados atuais:
Id: 2
Nome: Adidas
CNPJ: 11.111.111/0001-11

Informe os novos dados:
Nome: Nike
CNPJ: 11.111.111/0001-11
Pessoa alterada com sucesso.
```

Imagem 5 - Alteração de Pessoa

```
Output - CadastroPOO (run) X
1-Incluir Pessoa
2-Alterar Pessoa
3-Excluir Pessoa
4-Exibir pelo Id
5-Exibir Todos
6-Salvar Dados
7-Recuperar Dados
0-Finalizar Programa
Opção: 4
F-Pessoa Física | J-Pessoa Jurídica: j
Informe o Id: 2
Id: 2
Nome: Nike
CNPJ: 11.111.111/0001-11
```

Imagem 6 - Exibir pelo ID

```
Output - CadastroPOO (run) X
1-Incluir Pessoa
2-Alterar Pessoa
3-Excluir Pessoa
4-Exibir pelo Id
5-Exibir Todos
6-Salvar Dados
7-Recuperar Dados
0-Finalizar Programa
Opção: 3
F-Pessoa Física | J-Pessoa Jurídica: j
Informe o Id: 2
Pessoa excluída com sucesso.

1-Incluir Pessoa
2-Alterar Pessoa
3-Excluir Pessoa
4-Exibir pelo Id
5-Exibir Todos
6-Salvar Dados
7-Recuperar Dados
0-Finalizar Programa
Opção: 4
F-Pessoa Física | J-Pessoa Jurídica: j
Informe o Id: 2
Pessoa jurídica não encontrada.
```

Imagem 7 - Excluir pelo ID

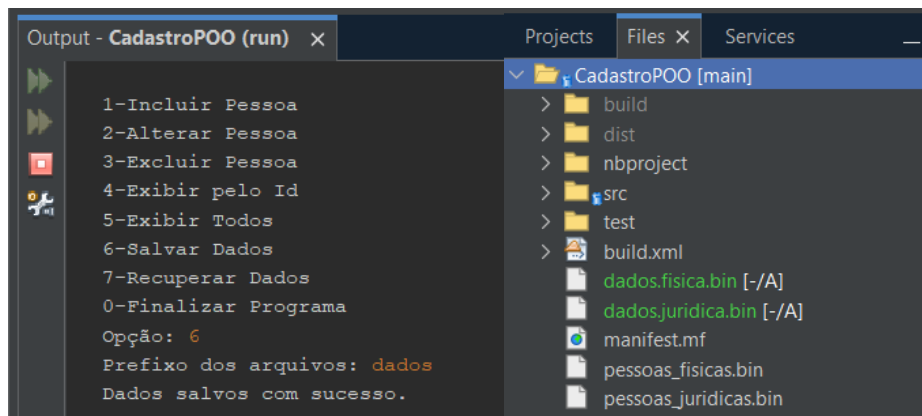


Imagem 8 - Salvando os dados em arquivo externo

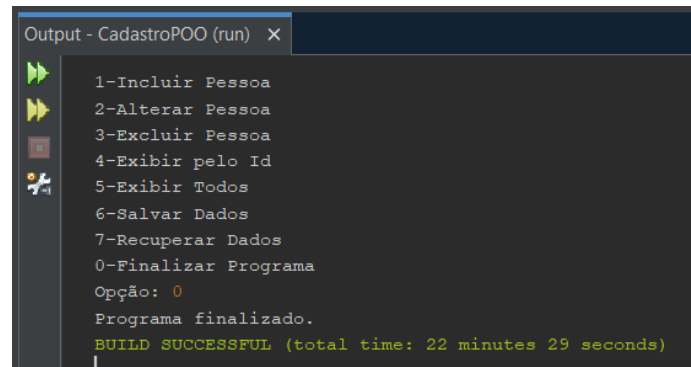


Imagem 9 - Finalização do programa

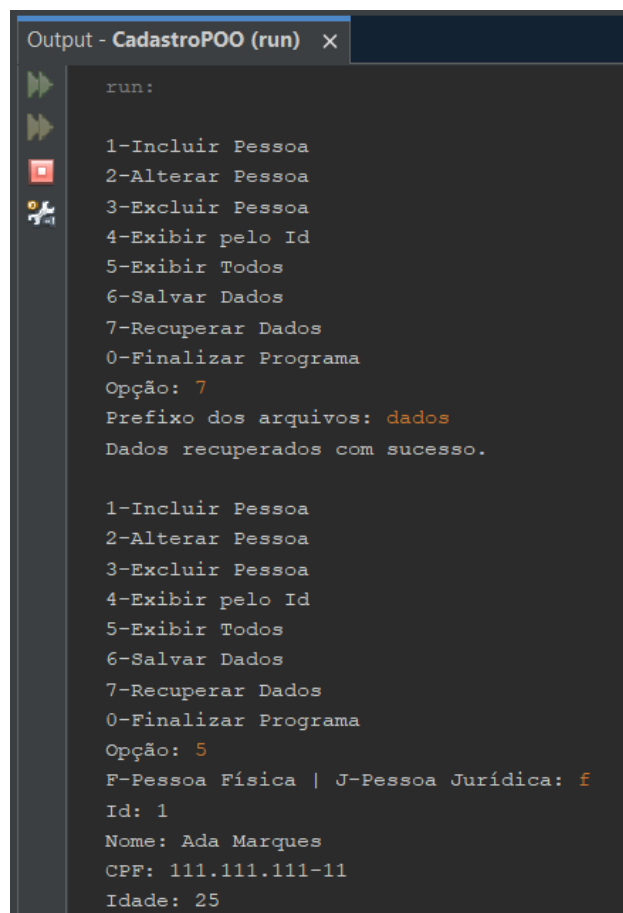


Imagem 10 - Recuperação dos dados salvos em arquivo externo

2.4. Análise e Conclusão

- O que são elementos estáticos e qual o motivo para o método main adotar esse modificador?

Elementos estáticos pertencem à classe e podem ser utilizados sem criar um objeto. O método main é estático porque precisa ser executado diretamente pela JVM para iniciar o programa.

- Para que serve a classe Scanner?

A classe Scanner serve para receber e interpretar os dados digitados pelo usuário. Nesta prática, ela foi usada para ler as opções do menu e os dados das pessoas.

- Como o uso de classes de repositório impactou na organização do código?

As classes de repositório concentraram as operações de cadastro, consulta e persistência dos dados. Isso deixou a classe principal mais organizada e separou as responsabilidades do sistema.